Enabling Information Centric Networks through Opportunistic Search, Routing and Caching *

Guilherme Domingues[1], Edmundo de Souza e Silva[1], Rosa Leão[1], Daniel S. Menasché[1]

[1]Universidade Federal do Rio de Janeiro (UFRJ)
Rio de Janeiro, RJ – Brasil

Abstract. *Content dissemination networks are pervasive in todays' Internet. Examples of content dissemination networks include peer-to-peer networks (P2P), content distribution networks (CDN) and information centric networks (ICN). In this paper, we propose a new system design for information centric networks which leverages opportunistic searching, routing and caching. Our system design is based on an hierarchical tiered structure. Random walks are used to find content inside each tier, and gateways across tiers are used to direct requests towards servers placed in the top tier, which are accessed in case content replicas are not found in lower tiers. Then, we propose a model to analyze the system in consideration. The model yields metrics such as mean time to find a content and the load experienced by custodians as a function of the network topology. Using the model, we identify tradeoffs between these two metrics, and numerically show how to find the optimal time to live of the random walks.*

1. Introduction

In today's Internet, there is a strong demand for content dissemination networks, such as *social networks* (e.g. Facebook) and *video networks* (e.g. Youtube and RIO [de Souza e Silva et al. 2006]). To support this growing demand, two broad classes of solutions, supported by IP host-to-host communications, have been proposed: Content Delivery Networks (CDNs) and Peer-to-Peer Networks (P2P).

In CDNs, *dedicated servers* store all the information published. Copies of popular contents are placed close to the users within cache servers. Users are automatically and transparently re-directed to the most appropriate server by a central authority. Quality of Service (QoS) and advanced monitoring techniques are deployed, through proprietary solutions, to redirect each user requests (e.g., Akamai Networks). In CDNs, a set of *origin servers* store a copy of all published content. Popular contents are also stored in *cache servers* close to users, so as to minimize the service delay experienced by the requesters. Requests for content are sent through IP routers to the central authority. Content flows back to users, along the IP routers.

In P2P systems, *peers* act as both client and servers for contents. Peers' requests are sent to other peers, from where content fragments (*chunks*) can be retrieved. Bittorrent and Emule are examples of P2P systems. As soon as peers have downloaded all desired content chunks, they may either leave the system or remain as future providers for chunks (*seeders*). While seeders, P2P nodes can leave the system for different reasons (*e.g.*, closing their connection to other peers, power failures, mobility). Indeed, there are no

*This research was supported in part by grants from CNPq and FAPERJ.

guarantees on the time a seeder will remain present in a P2P system. If peers leave the system immediately after concluding their downloads, content might become unavailable which might lead to instability [Menasché et al. 2010]. Peers exchange information through IP routers, but in contrast to CDNs, there are no dedicated servers to cache content close to users.

According to [Ahlgren et al. 2012], global traffic in the Internet will increase by a factor of four from 2009 to 2014, approaching 64 exabytes per month in 2014, compared to approximately 15 exabytes per month in 2009. Global mobile data traffic is expected to double every year through 2014. Despite the tremendous success of CDNs and P2P systems, they present issues if we consider the *scalability* and *reliability* envisioned for next generation of content dissemination networks. With billions of mobile nodes claiming for contents, central decision policies at CDNs tend not to scale. In P2P networks, the absence of dedicated cache servers close to users, as well as the lack of guarantees for a peer to remain in the system after obtaining the desired content, hampers reliability.

The scalability and reliability challenges for massive distribution content gave rise to a new research area: Information Centric Networks (ICN) [Ghodsi et al. 2011]. In ICNs, users know the *name* of a content being searched, before issuing requests. All searched content is previously stored in the network through subscriptions to the network. Caching at all routers, also referred to as *universal caching*, is considered. Under universal caching, caching is provided to all users, and can be potentially implemented by all routers, being pervasive along the entire network. Figure 1 exemplifies this scenario. Questions on how to deploy efficient algorithms to explore *universal caching* and on how to define *inter-domain* routing policies give rise to important challenges.

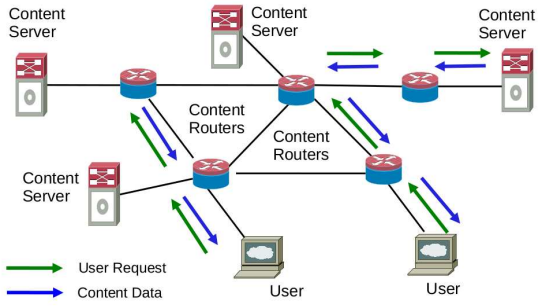


Figura 1. ICN scenario

Figure 2 shows some of the most important ICN features. In ICNs, content is published without any explicit destination address. Receivers subscribe to the network through a query (request) for named contents. Users' requests are forwarded by structured or non-structured topologies. When there is a matching between a query and a stored content, content is delivered to the subscribers. During delivery, content might be cached and replicated in the network. This strategy is known as *in-networking caching*. Content can be sent to subscribers over TCP/IP transport mechanisms over paths which are oblivious to the path taken by the requests. Alternatively, content can be sent in the reverse path of requests trails stored across the caches. We reference caches in ICNs henceforth as *cache-routers*.

In this article, our main contribution is a novel ICN architecture with routers dis-

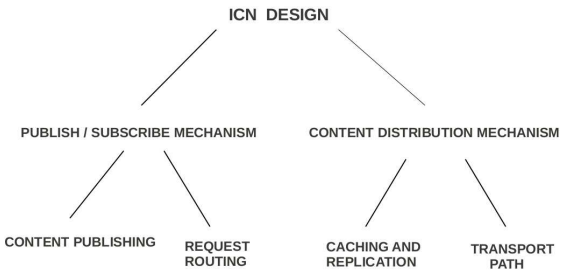


Figura 2. Publish/subscribe and content distribution mechanisms

posed along hierarchical tiers domains, without a global lookup service to map names to IP addresses to all content present in the network:

• **System design:** We propose an opportunistic search algorithm to be executed across hierarchical tiers. Random walks are used to opportunistically find replicas of the content inside a tier. If no replicas are found, requests are forwarded to another tier in the direction of publish area servers, which maintain fixed copies of the content. This way, we cope with the tradeoff between exploration of new routes to content replicas and exploitation of known routes to fixed replicas. In addition, we also propose an opportunistic caching policy using reinforced counters. The content placement and eviction policies are targeted towards self-tuned storage mechanism which must distribute content in the network to satisfy demands that vary over time.

• **Analytical model:** We propose an analytical model to evaluate ICNs inspired by reliability theory concepts. The model yields metrics such as the mean time to find a content and the load at the publishing areas as a function of the network topology and the time to live (TTL) of requests inside domains. Our model allows us to study tradeoffs in the choice of the TTL. Smaller values of TTL might lead to a reduction in the time to retrieve information at the cost of an increase in the system's load at publishing areas.

The remainder of this paper is organized as follows. Section 2 presents related work and a background on Information Centric Networks. Section 3 introduces the proposed ICN system design, Section 4 contains the analytical model and in Section 5 we present numerical results obtained with the proposed model. Section 6 concludes the paper.

2. Related Work

The literature on ICN can be broadly classified into structured architectures, such as [Koponen et al. 2007, Lagutin et al. 2010, 2nd NetInf Description 2010] and unstructured architectures, such as [Rosensweig et al. 2010, Kakida et al. 2011]. In what follows, we briefly describe some of the proposals.

Dona [Koponen et al. 2007] consists of an hierarchy of domains, wherein a resolution handler (RH) knows the IP location of all content published in descendent domains. RHs placed in the highest domain are aware of all the content published in the entire network. Psirp [Lagutin et al. 2010], Netinf [2nd NetInf Description 2010] and Mulitcache [Katsaros et al. 2011] are proposals that deploy Distributed Hash Tables (DHT), enabling multiple logical entities to share the knowledge of all content published in the

network, in a distributed fashion. Each content placed in the network is mapped to a hash key, with a hash key being associated with a single node of the DHT.

DHT structures are highly scalable, but may impose considerable overhead maintenance costs [Chen et al. 2008]. Whenever a node fails or becomes repaired, requests must be sent along the network to reassure the node location in the network. Neighborhood adjacencies must be reestablished and file-index databases related to a partition management must be resettled. Also, many unsolved security vulnerabilities are able to disrupt the pre-defined operation of DHTs nodes [Urdaneta et al. 2011]. At last, in a network composed of domains where providers care about administrative autonomy, the use of a global hash table is unfeasible [D'Ambrosio et al. 2011].

Jacobson et al. [Jacobson et al. 2009] propose a non structured geometry topology for routing requests. Published content is announced through routing protocols, composing routing tables supporting name aggregation. Requests are routed towards publishing areas leaving a trail called *bread crumbs*, so content follows the reverse paths set by the trails, when sent to users. As content flows to users, the bread crumbs are consumed. In general, content replicas will not be stored in the domain they were originally published. To be encountered, routes for replicas should be in the routing tables. Avoiding an explosion of the size of the routing tables becomes an important challenge.

To enhance the discovery of cached contents, Rosensweig et al. [Rosensweig et al. 2010] and Kakida et al. [Kakida et al. 2011] allow bread crumbs not to be consumed on the fly, when content traverses the network. This allows trails for previously downloaded contents to be preserved. A new user request that reaches a trail for a desired content will be sent to a cache-router indicated by the trail, before flowing to the publishing areas. Opportunistically downloading content from nearby cache-routers may considerably reduce the time users take to retrieve information. Nevertheless, the proposals encompassing the preservation of bread crumbs do not explicitly address how published information will be announced to fulfill the routing tables.

In this article, we avoid the drawbacks mentioned in this section, for both structured and unstructured geometry topologies. To this aim, we propose a novel architecture with cache-routers disposed across hierarchical domains. Users' requests flow across tiers towards the publishing areas. Random walks are issued to explore domains' vicinity, so as to allow opportunistic encounters with the desired content. That way, we avoid the drawbacks of structured geometries, as well as the problem of fulfilling routing tables which is common to the unstructured architectures discussed above.

The model presented in this paper is inspired by reliability metrics [de Souza e Silva and Muntz 1992]. The performance of random walks for search in unstructured hybrid peer-to-peer systems has been analyzed by Ioannidis and Marbach [Ioannidis and Marbach 2008]. Nonetheless, Ioannidis and Marbach focus on asymptotic results when the number of nodes in the network grows to infinity, and considered a single domain (tier). To the best of our knowledge, our work is the first to analyze the performance of random walks for search in multi-tiered architectures, accounting for the tradeoffs involved in the choice of the time to live of the random walks. Existing multicache multi-tier normal cache overlay network either consider global knowledge of content placement (CDN, P2P), or single path to the storage area (Cache Trees). None of these methods

explore the vicinity of a cache in a fully distributed way.

3. System Design

Our architecture design considers **logical** hierarchical *tiers* (also called *domains*) composed of *cache-routers* ($i.e.$, routers that can store content replicas). We consider N logical hierarchical tiers, in which tier 1 is the top level tier, and tier N constitutes the bottom level. Routers in tier N are the only ones connected to users. Users publish content in storage areas connected to tier 1 routers. The interaction among the publishing areas and tier 1 routers is better detailed in Figure 3. The figure displays a single publishing area in tier 1 and the logical hierarchy of cache-routers. Routers forward requests towards publishing areas which contain permanent copies of the content. Replicas may be cached in the cache-routers in the logical hierarchy. A strategy should be adopted to allow *opportunistic* encounters between requests and replicas in a best-effort manner as will be detailed below. Opportunistic encounters are probabilistic in nature.

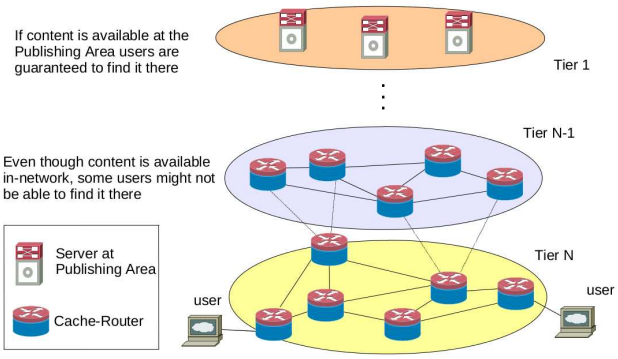


Figura 3. ICN content distribution mechanisms.

3.1. Content search: random walks and bread crumbs

When requests are issued by users the cache-routers forward them to the publishing areas. Each cache-router of tier i is logically connected to a subset of cache-routers in the same tier i and in the parent tier $i-1$. When a request first reaches a tier, a random walk is issued to find replicas of the content that may have been cached in that tier. The random walk lasts for at most T units of time, and only traverses cache-routers in the same level. That is, a counter is set to limit the amount of search time for a content *within a level*. If the desired content is not found when this counter expires, the router (say router k at level i) that holds the request at that time transfers it to level $i-1$, that is, to one of the routers in level $i-1$ router k is (logically) connected to. A new random walk will then be issued, but now at level $i-1$. If the content been searched is found at a cache-router of hierarchy j, it is sent to the user as it will be explained below. Otherwise, the request will be forwarded up in the hierarchy until the publishing area (tier 1) is reached where the content is guaranteed to be found (otherwise the content was never stored in the first place). Note that the search does not generate significant traffic. Each request from a user generates a single search message that performs a random walk at one tier at a time. The traffic due to this request message is negligible as compared to that generated by the content to be transferred.

As the requests are forwarded upwards across the hierarchy, a set of *b*ackward pointers is maintained. Those pointers are henceforth referred to as *bread crumbs*. When a content is located in the network, it is delivered to the users following the reverse path of the bread crumbs. As the content follows the path of *bread crumbs*, the trail is erased. Random walks within domains do not generate bread crumbs. Therefore, when content is delivered to users it does not follow the exact same path as the random walks. This means that random walks do not impose extra hops/side effects when content is delivered to users. We use a set of counters, named *reinforced counters* (RCs), to keep track of the load across the network for each content. The placement of replicas will be controlled using the reinforced counters, which will be used to decide which and when contents should be stored or evicted at each cache-router. Our approach is self-adaptive to users' loads.

3.2. Content placement: reinforced counters

Each request carries a hash key *inf* to identify the content being searched for. As mentioned above, a trail of bread crumbs is left at each cache-router traversed by the request. The trail includes a back pointer to the preceding router visited by the request (the bread crumb), the hash key *inf* to identify the request and an associated counter (called *reinforced counter*) that is incremented when the request arrives at a cache-router. The *reinforced counter* is used to determine whether the content associated with inf should be stored at the cache-router. Each cache-router keeps a reinforced counter rc(*inf*), associated to each hash key *inf*. Whenever a request for *inf* reaches a cache-router, rc(*inf*) is incremented by one. When content is downloaded through the reverse path left by the trails, bread crumbs are consumed but the reinforced counters are kept intact. Each cache-router periodically decreases the reinforced counter rc(*inf*) associated to *inf* and this period is a parameter to be set.

A reinforced counter rc(*inf*) for content *inf* at a cache-router has two thresholds: rc-up(*inf*) and rc-low(*inf*). If rc(*inf*) reaches the upper threshold rc-up(*inf*) the content *inf* is cached at that cache-router after it is found and when it traverses the cache-router towards to the user. Whenever rc(*inf*) reaches the lower threshold rc-low(*inf*) and if *inf* is cached at that cache-router, this content is immediately removed from the cache-router. We assume that cache-routers have enough storage capacity to store any content that they are required to maintain. The reinforced counters mechanism allows popular contents to be transferred from the publishing areas to the cache-routers. That way, the opportunistic discovery of popular content is favored at cache-routers. In contrast, if demand is not high enough to justify consumption of storage resources, reinforced counters will favor the eviction of the content.

We provide additional insights on the connection between reinforced counters and content placement. For simplicity of exposition, and without loss of generality, we assume that content is demanded by users at a given fixed rate. We then show that the reinforced counters fully determine content placement at the cache-routers. To this aim, we consider a fluid approximation, where content requests arrive according to a flow with a given intensity, and the reinforced counters are also approximated as being continuous. Let $\Lambda_{m,l}(\mathit{inf})$ be the total load for *inf* arriving from tier $l+1$ at the m-th cache-router at tier l. Let $\gamma(\mathit{inf})$ be the rate at which the reinforced counters rc(*inf*) are decremented. We assume that the rc(*inf*) are initialized as rc-low(*inf*), so that if $\Lambda_{m,l}(\mathit{inf}) = \gamma(\mathit{inf})$ then

content will not be stored at m.

Proposition 3.1. *Under the fluid approximation and an hierarchical tiered topology, a cache-router stores a copy of* inf *if and only if* $\Lambda_{m,l}(\mathsf{inf}) > \gamma(\mathsf{inf})$.

Proof: The proof is inspired by [Rosensweig et al. 2013, Lemma 1]. In an hierarchical network we define the direction of requests as *upstream* and the reverse as *downstream*. A cache-router is affected only by requests that pass through it, and requests pass through routers only upstream, so upstream cache-routers do not impact cache-router m through their requests. Content flows downstream, but only passes through m for requests that were issued by cache-router m. Therefore, the state and requests of upstream routers do not impact the state and requests issued by downstream routers.

We can determine if a cache-router will store a content in a bottom-up manner. First, consider the leafs of the tiered topology. At such cache-routers, if $\Lambda_{m,l}(\mathit{inf}) > \gamma(\mathit{inf})$ then rc(*inf*) will eventually reach rc-up(*inf*), and content will be stored and never evicted. If $\Lambda_{m,l}(\mathit{inf}) \leq \gamma(\mathit{inf})$, in contrast, rc(*inf*) will remain equal to its initial value rc-low(*inf*). rc(*inf*) will afterwards remain fixed, and the cache-router, marked. Once leafs are marked, we proceed upstream. Using the same argument as the one in the paragraph above, we determine the content placement at the next cache-router that has all its children marked, and so on, up to reaching routers at tier 1, where all content is stored. □

Given a workload, proposition 3.1 can be used to determine which domains across the network will hold a copy of each content, if we know the (probabilistic) choice by which a router of level i passes requests to routers of level $i-1$ at which it is logically connected to. In the remainder of this paper we will assume that the probability distribution of content over cache-routers is fixed and given for all cache-routers.

4. System Analysis

In this section we present the model analysis for a single domain (i.e., a logical tier). Our main goals are to derive the key metrics of interest, namely 1) mean time to find content and 2) the fraction of requests that hit the publishing area. These metrics are important to evaluate the performance of the proposed architecture. We consider a network of C cache-routers in a domain. Figure 4(a) illustrates a domain with five cache-routers. Assume that the desired content is stored at cache-routers 3, 4 and 5. Suppose that a request arrives from a downstream domain to cache-router 2. Then, cache-router 2 starts a random walk in the domain to find the content. Let T be the maximum time a request might spend in a domain before being redirected to the upstream domain in the hierarchy. T is also referred to as time to live, or TTL.

We consider a continuous time random walk in which the time between walker movements are exponentially distributed with rate ψ, with the associated infinitesimal generating matrix Q. Let $\Delta(i)$ be the out-degree of node i and ψ the walker rate. Then, all diagonal elements in Q are equal to $-\psi$, and $q_{ij} = \psi/\Delta(i)$ if there is a logical link from i to j. We use uniformization [de Souza e Silva and Gail 2001] to obtain the metrics of interest. Let P be the *uniformized matrix*, obtained from Q as $P = I + Q/\Lambda$, where Λ is a positive number greater than the maximum absolute value of the elements in the diagonal of Q. Λ is also referred to as the *uniformization rate*.

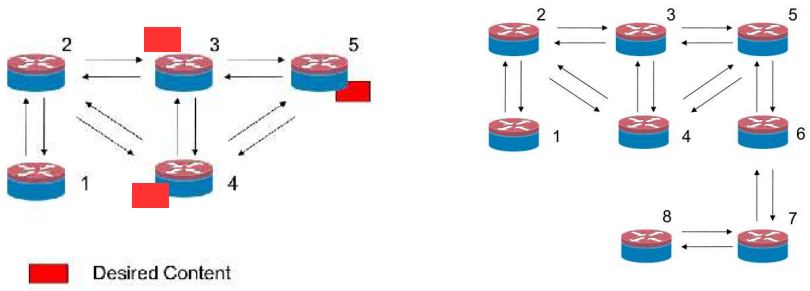
variable	description
C	number of cache-routers in the network
T	time to live
$H(T)$	time to hit content
$L(T)$	lifetime of random walk in a domain
$R(T)$	probability of not hitting content by T
T_0	time cost for accessing the publishing area
p	probability that content is in domain

Tabela 1. Table of notation. Time to live, T, is integer when considering discrete time random walks, and real when considering continuous time random walks.

Let $\Omega_P(k)$ be the probability that, after k transitions of the uniformized process, the random walk had not hit the desired content. To compute $\Omega_P(k)$ we construct a modified transition matrix $\widetilde{P}$, assuming that the desired content is placed in a subset of cache-routers of the domain. Let $\boldsymbol{\omega}$ be a column vector of size C where $\boldsymbol{\omega}_i = p_i$ and p_i is the probability that cache-router i stores the content, $\sum_{i=1}^{C} p_i = 1$. To facilitate the notation, we number the cache-routers where $p_i > 0$ with the highest indexes of matrix P. In Figure 4(a) they are numbered 3,4 and 5. Then, $\widetilde{P}$ is defined as

$$\widetilde{P} = P(I - (diag(\boldsymbol{\omega}))) \tag{1}$$

where $diag(\boldsymbol{\omega})$ is a matrix with the diagonal elements equal to vector $\boldsymbol{\omega}$ and all other elements equal to zero. $\widetilde{P}$ is a sub-stochastic matrix where the sum of the elements in line i is the probability of not finding the content in the domain, in one step, given that the random walk starts at cache-router i. The element (i, j) of $\widetilde{P}$ is the probability that a random walker that starts at cache i moves to cache j in one step and then a cache miss occurs at j.



(a) (b)

Figura 4. (a) Example of a domain with five cache-routers and (b) Example of a domain with eight cache-routers.

Let $\pi(0)$ be the initial state probability vector indicating from where the search starts in the domain. The i-th element of $\pi(0)$, $\pi_i(0)$, corresponds to the probability that the random walk starts at cache-router i. The (i, j) entry of matrix $\widetilde{P}^k$ characterizes the probability of visiting state j after jump k, given that the system starts at state i. Let $\boldsymbol{\upsilon}_{\widetilde{P}}(k)$ be a vector whose m-entry is the probability that after k steps a miss occurs at

cache-router m, given that the random walk initial state is $\pi(0)$. The following recursion can be used to compute $\boldsymbol{\upsilon}_{\widetilde{P}}(k)$,

$$\boldsymbol{\upsilon}_{\widetilde{P}}(k) = \boldsymbol{\upsilon}_{\widetilde{P}}(k-1)\widetilde{P}, \qquad k > 0 \tag{2}$$

where $\boldsymbol{\upsilon}_{\widetilde{P}}(0) = \pi(0)$. Then, $\Omega_P(k)$ is given by

$$\Omega_P(k) = \boldsymbol{\upsilon}_{\widetilde{P}}(k)\mathbf{U}, \qquad k \geq 0 \tag{3}$$

where $\mathbf{U}$ is a column vector with all elements equal to one.

Mean Time to Reach Content

Recall that T is the maximum time to live of a random walk in a domain, $T \in \mathbb{R}$. Let $R(T)$ be the probability that the content is not found by T units of time. Then, conditioning on the number of transitions in the interval $[0, T]$,

$$R(T) = \sum_{n=0}^{\infty} \varphi(n, T)\Omega_P(n) \tag{4}$$

where $\varphi(n, T) = \exp(-\Lambda T)(\Lambda T)^n/n!$ is the probability that n jumps occur in the interval $[0, T]$. The number of jumps in the interval $[0, T]$ is Poisson distributed as we uniformized Q.

Next, our goal is to compute the expected hitting time of a content, $E[H(T)]$. The expected hitting time of a content is the sum of two components. The first component characterizes the mean time to hit the content given that it is available in the domain, while the second component characterizes the mean time to hit the content if it is unavailable in the domain. In case the content is available in the domain, the *lifetime of the random walk* in the domain (i.e., the searching time in the domain), $L(T)$, is the minimum between the time until reaching the content and T. The mean of $L(T)$ is given by

$$E[L(T)] = \int_0^T R(t)dt \tag{5}$$

where $\lim_{T\to\infty} E[L(T)]$ is referred to as *mean time to exit* in the reliability literature, and can be evaluated using standard techniques [de Souza e Silva and Gail 2000].

Let p be the probability that the content is available in the considered domain. If the content is available, the probability that the content is not found by time T is $R(T)$. If the content is unavailable, the random walk will take time T inside the domain, and additional T_0 units of time to reach the publishing area. Therefore, the expected hitting time of a content is

$$E[H(T)] = (E[L(T)] + T_0 R(T))\, p + (T_0 + T)(1 - p) \tag{6}$$

Replacing (4) into (6), and exchanging the order of the integration and summation, yields, after simplification,

$$E[H(T)] = \left(\sum_{n=0}^{\infty} \left(1 - \sum_{m=0}^{n} e^{-\Lambda T} \frac{(\Lambda T)^m}{m!} \right) \Lambda^{-1} \Omega_P(n) + T_0 R(T) \right) p + (T_0 + T)(1 - p) \tag{7}$$

The analysis above easily handles discrete time random walks, where time between walkers movement is fixed.

5. Experimental Results and System Analysis

In this section we present numerical results for a few examples in order to illustrate our proposal and the existing tradeoffs in the choice of parameter values. In the development of section 3, the time intervals between the random walker steps can be either independent and exponentially distributed random variables or constant. For the numerical studies we consider constant step values.

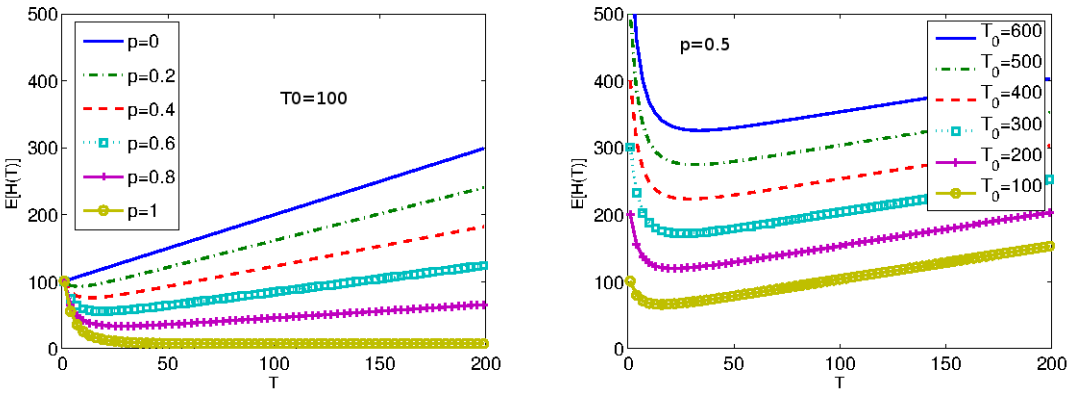
Our goals are to 1) show the impact of different system parameters on the mean time to find content and 2) evaluate the average load on publishing area servers, which impacts both the time to recover a content and the system's scalability. We analyze a simple two-tier setup wherein the publishing area servers have finite capacity. This simple architecture suffices to highlight the key points we want to emphasize in this work while keeping the model description short. From this scenario we also indicate how the evaluation could easily consider multiple logical tiers.

Our reference topology is that illustrated in Figure 4(a). In this topology, there is a single content that may be cached in one of the cache-routers $3, 4$ or 5 with equal probability of residing in any of these three routers, given it is in this logical tier. Therefore, $p_u = 1/3$ for $u = 3, 4, 5$. With probability $1 - p = 0.5$, the content does not reside in the tier and it takes and additional $T_0 = 100$ time units to find it in the publishing area. The parameter T, the maximum time the random walker is allowed to search for the content in a logical tier is varied between 1 and 200. The parameters values may vary according to the experimental goals.

5.1. Single Domain Case

We illustrate how the mean time to find the content, $E[H(T)]$, depends on different system parameters. Figure 5(a) shows $E[H(T)]$, obtained using (6), as a function of T, for different values of p varying between 0 and 1, with increments of 0.2. The topology and the content placement are kept fixed. Let $T^\star$ be the optimal value of T that minimizes $E[H(T)]$. As the probability that the content is available increases, $T^\star$ increases. Clearly, when $p = 0$, $T^\star = 0$, since if the content is not available in the tier it is better not to search for it. On the other hand, when the content is always available ($p = 1$), $T^\star \to \infty$. This is because, in this example, T_0 is large compared to the mean time to find the content in the tier ($E[H(\infty)]$). In this scenario, it will take less time, on average, for the random walker to find the content in the domain, as opposed to find it in the publishing area servers, if the walker is given enough time to search for the content in the domain (i.e., if T is very large). For values of p between $(0, 1)$, the optimum value $T^\star$ can be obtained from our model as shown in Figure 5(a).

For any given value of p, Figure 5(a) shows that $E[H(T)]$ first decreases and then increases, except for $p = 0$ and $p = 1$, when $E[H(T)]$ always increases and decreases, respectively. In any case, $E[H(T)]$ asymptotically grows at rate $1 - p$, as can be inferred from equation (6). The impact of T_0 (the time cost for accessing the publishing area servers) on $E[H(T)]$ is illustrated in Figure 5(b), which shows $E[H(T)]$ as a function of T, for $T_0 \in [100, 200, \ldots, 600]$. As T_0 decreases, there is a subtle decrease of the optimal time to find a content in the domain. This is because, as the time to access publishing area servers increases, it is beneficial to remain longer in the domain before forwarding the request to other domains.



(a) (b)

Figura 5. Effect of p and T_0 on the expected hitting time

Until now we analyzed the problem of content discovery from the clients perspective. The administrators of publishing areas, in turn, might worry about the traffic that is incurred in their servers. If content is available in the cache-routers in a tier, the load from that domain to the servers (assuming a single tier architecture) is proportional to $R(T)$, the probability that users do not find the content in the tier by time T. In more detail, let λ and Λ be the average user load (number of requests for a content per time unit) that arrives to a tier and the average load flowing from the tier to the next in the hierarchy (or from the tier to the publishing area), respectively. The number of requests per unit time served inside the domain is clearly $\lambda - \Lambda$ and $\Lambda = R(T)\lambda$. For the plots we consider $\lambda = 100$.

Figures 6(a) and 6(b) show how Λ varies as a function of T. Figures 6(a) considers the reference scenario, modified so that content may be available at caches 2, 3, 4 and 5 with probability 0.25 at each, given the desired content is in the domain. Figures 6(b), in turn, is generated from the topology with 8 caches shown in Figure 4(b), where content may be available at caches 5, 6, 7 and 8 with probability 0.25 at each, given the desired content is in the domain. The other parameters are the same as in the reference scenario.

Figures 6(a) and 6(b) indicate that there is a tradeoff between users interests and providers time-cost when choosing the optimal value of T. If T is selected so as to minimize $E[H(T)]$, the rate at which the publishing area servers are accessed, Λ might be considerably large. However, a slight increase in T and $E[H(T)]$ might yield significant reductions in Λ. Therefore, we may minimize $E[H(T)]$ constraining Λ to a given (relatively small) value, that is, we may be able to keep the load at the publishing area servers at a low value and yet avoiding a considerable increase in the time to retrieve the searched content.

Note that in Figure 6(b) Λ is constant for $T \leq 3$. This is because we are considering a discrete time random walk for the studies. In this case, the probability that the content is found inside the domain is zero if T is smaller than the distance between the source of the random walker and the cache-router that may have the content. In a continuous time random walk, in turn, $R(T)$ is strictly decreasing with respect to T.

We may summarize the above findings as follows. If content is available in a tier with high probability, it may be advantageous to increase the time the random walker is

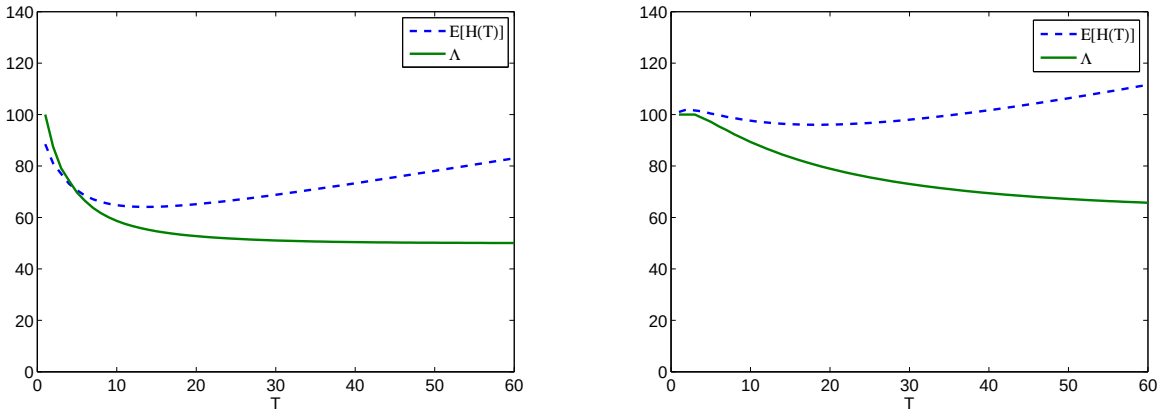


Figura 6. Probability of not hitting content by time T, $E[H(T)]$, and Λ, for (a) 5 cache topology, (b) 8 cache topology.

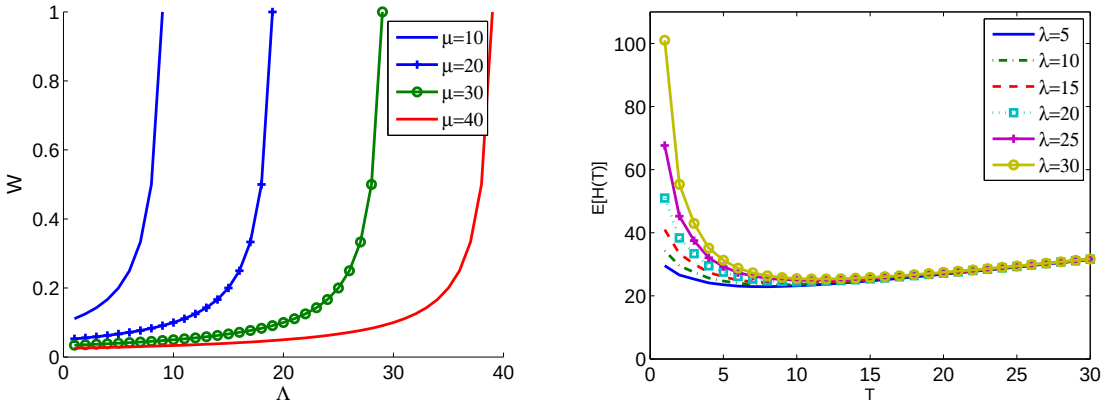
allowed to search for a content in the tier to reduce the load at the publishing area servers. On the other hand, the expected time to find the content may increase. Quantifying these tradeoffs is a contribution of our work.

5.2. Multiple Domain Case

In the previous section we assumed a constant time T_0 to retrieve a content from the publishing area servers. In what follows we consider a scenario where content that is not found in a domain is forwarded to publishing area servers with limited capacity. Therefore, the time cost to access the publishing area servers, T_0, will now be a function of the amount of traffic Λ that is directed to the publishing area servers (which in turn is a function of $R(T)$). To illustrate the impact of the load of users on the publishing area servers, let us a consider a simple M/M/1 model for the server, with associated service capacity μ. Then, the mean waiting time experienced by users accessing the server is $W = 1/(\mu - \Lambda)$. Figure 7(a) shows how W varies as a function of Λ. In what follows, for illustration purposes, we set $\mu = 40$ and $T_0 = 1000W$,

$$T_0 = 1000/(\mu - R(T)\lambda) \tag{8}$$

Figure 7(b) illustrates the impact of the service capacity on the mean waiting time experienced by users and shows how $E[H(T)]$ varies as a function of T. The qualitative behavior of $E[H(T)]$ is the same as the one observed previously, and the insights obtained in Section 5.1 about the tradeoff in the choice of T still apply. Note that an increase in $R(T)$, the probability of not finding the searched content in the tier (not shown in the figure) may degrade users performance, as it may lead to server overload. In particular, if T is small (e.g., $T = 1$), $E[H(T)]$ grows unbounded with λ. Due to space limitations we were only able to discuss a few examples aiming at describing our proposal and the existing design tradeoffs. Our approach, however, can handle multiple domains and user loads as we briefly sketch below. From the results of section 4 and given the user loads, we can obtain the domains that hold copies of the content. $R(T)$ is then used to obtain the loads at each domain as well as at the publisher. Finally, given the loads at the different domains, $E[H(T)]$ is obtained in a top down fashion, starting from the top and moving to the lowest level domains.



(a) (b)

Figura 7. (a) Average server processing time as a function of Λ, (b) mean time to find content, accounting for server capacity $\mu = 40$.

6. Conclusion

Information centric networks are gaining considerable attention from researchers and practitioners due to their scalability and robustness properties. Nonetheless, there are still important challenges in the design, modeling and analysis of such networks. From the design perspective, one of the challenges is to determine how to fill out the routing tables. From the modeling and analysis perspective, the challenges are associated to the large number of routers envisioned in ICNs. In this paper, we have proposed a novel design for ICNs that is built on top of random walks and hierarchical tiers (domains) to cope with the exploration *versus* exploitation tradeoff involved in content search.

Our model computes metrics such as mean time to find a content and evaluate existing tradeoffs to tune the parameters of the architecture we propose. In this present work we focus on the performance tradeoffs of the architecture. However, our analytical model can be extended to study the scalability of the proposed architecture with respect to others in the literature. For instance, as the user load for a content grows what is the overall increase in the expected time to find a content as compared to a P2P architecture. This work also opens additional avenues for future research. One such problem is to determine the impact of parallel random walks across tiers at the same level of the hierarchy as well as across different levels.

Referências

2nd NetInf Description (2010). Netinf. `http://www.4ward-project.eu/`.

Ahlgren, B., Dannewitz, C., Imbrenda, C., Kutscher, D., and Ohlman, B. (2012). A survey of information-centric networking. *IEEE Communications Magazine*, 50(7):26–36.

Chen, S., Zhang, Z., Chen, S., and Shi, B. (2008). Efficient file search in non DHT P2P networks. *Elsevier Computer Communications*, 31(2):304–317.

D'Ambrosio, M., Dannewitz, C., Karl, H., and Vercellone, V. (2011). MDHT: A hierarchical name resolution service for information-centric networks. In *Proc. of SIGCOMM workshop on ICN*, pages 7–12.

de Souza e Silva, E. and Gail, H. R. (2000). Transient Solutions for Markov Chains. In Grassmann, W., editor, *Computational Probability*, pages 44–79. Kluwer.

de Souza e Silva, E. and Gail, H. R. (2001). The Uniformization Method in Performability Analysis. In B. R. Haverkort, R. Marie, G. R. and Trivedi, K. S., editors, *Performability Modelling: Techniques and Tools*, chapter 3, pages 31–58. Wiley.

de Souza e Silva, E., Leão, R. M. M., Santos, A. D., Azevedo, J. A., and Netto, B. C. M. (2006). Multimedia Supporting Tools for the CEDERJ Distance Learning Initiative applied to the Computer Systems Course. In *22th ICDE World Conference on Distance Education*, pages 1–11.

de Souza e Silva, E. and Muntz, R. R. (1992). *Métodos Computacionais de Solução de Cadeias de Markov: Aplicações a Sistemas de Computação e Comunicação*. SBC - Escola de Computação.

Ghodsi, A., Schenker, S., Koponen, T., Singla, A., Raghavan, B., and Wilcox, J. (2011). Information-centric networking: Seeing the forest for the trees. In *Proc. of HOTNETS*, pages 1–6.

Ioannidis, S. and Marbach, P. (2008). On the design of hybrid peer-to-peer systems. *ACM SIGMETRICS Performance Evaluation Review*, 36(1):157–168.

Jacobson, V., Smetters, D. K., Thornton, J. D., Plass, M. F., Briggs, N. H., and Braynard, R. L. (2009). Networking named content. In *Proc. of CoNEXT*, pages 1–12.

Kakida, M., Tanigawa, Y., and Tode, H. (2011). Breadcrumbs+ : Some extensions of naive breadcrumbs for in-network guidance in content centric networks. In *Proc. of IEEE IPSJ*, pages 376–381.

Katsaros, K., Xylomenos, G., and Polyzos, G. C. (2011). Multicache: An overlay architecture for information-centric networking. *Elsevier Computer Networks*, 55(4):936–947.

Koponen, T., Chawla, M., Chun, B. G., Ermolinskiy, A., Kim, K. H., Shenker, S., and Stoica, I. (2007). A data-oriented (and beyond) network architecture. In *Proc. of SIGCOMM*, pages 181–192.

Lagutin, D., Visala, K., and Tarkoma, S. (2010). Publish/subscribe for internet: PSIRP perspective. In *Emerging Trends from European Research, (Valencia FIA book 2010)*.

Menasché, D. S., Rocha, A. A., de Souza e Silva, E., Leão, R. M. M., Towsley, D., and Venkataramani, A. (2010). Estimating self-sustainability in peer-to-peer swarming systems. *Perf. Eval.*, 67(11):1243–1258.

Rosensweig, E., Kurose, J., and Towsley, D. (2010). Approximate models for general cache networks. In *Proc. of INFOCOM*, pages 1–9.

Rosensweig, E., Menasché, D., and Kurose, J. (2013). On the steady-state of cache networks. In *Proc. of INFOCOM, to appear*.

Urdaneta, G., Pierre, G., and Steen, M. V. (2011). A survey of DHT security techniques. *ACM Comp. Surveys*, 43(2).